

The ROS 2 Computation Graph

Nodes, messages, discovery, and the setting that decides whether anything arrives

Prof. Anis Koubaa Dr. Asem Ibrahim Alalwan

EE 414 — Introduction to Robotics
College of Engineering
Alfaisal University

Fall 2026 — Week 2, Lecture A



What this lecture does

Roadmap

Last week we argued that robot software must be many small programs exchanging messages. Today: what those programs are, what the messages are, how they find each other, and why a correct-looking pair sometimes exchanges nothing at all.

- 1 Nodes and the graph
- 2 Four ways nodes talk
- 3 Messages: the contract
- 4 How nodes find each other
- 5 Quality of Service
- 6 Next

Outline

- 1 Nodes and the graph
- 2 Four ways nodes talk
- 3 Messages: the contract
- 4 How nodes find each other
- 5 Quality of Service
- 6 Next

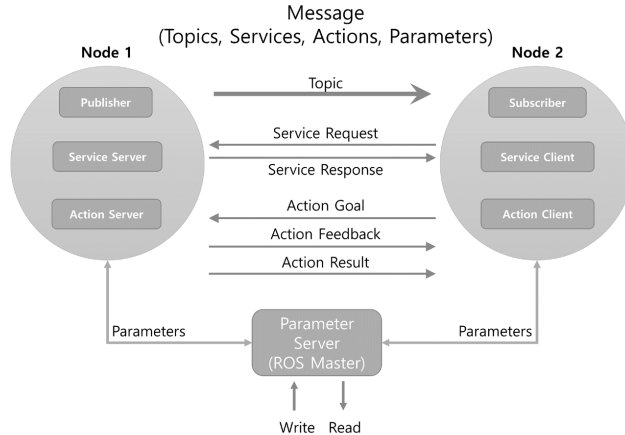
Where we left off

The naive robot program failed for five structural reasons:

The problem	The answer we are about to build
One rate for everything	Each node has its own timer
One crash kills everything	Nodes are separate OS processes
One machine	Nodes discover each other over the network
Nothing is reusable	Nodes agree on message <i>types</i> , not on each other
You cannot see inside it	Every message is inspectable from outside

Today is the vocabulary for all five. **Session B is where you type it.**

The whole lecture, in one picture



Two nodes. Four ways to talk. Nothing else — and we spend today on the four arrows.

Figure: *ROS Robot Programming*, Y. Pyo, H. Cho, L. Jung, D. Lim (ROBOTIS, 2017). Reused from the instructor's ROS course.

The node: the unit of a robot program

A **node** is one process that does one job.

Good nodes

- `lidar_driver` — talk to the LiDAR, publish scans
- `controller` — turn a path into wheel velocities
- `battery_monitor` — watch the voltage, warn

A node that will hurt you

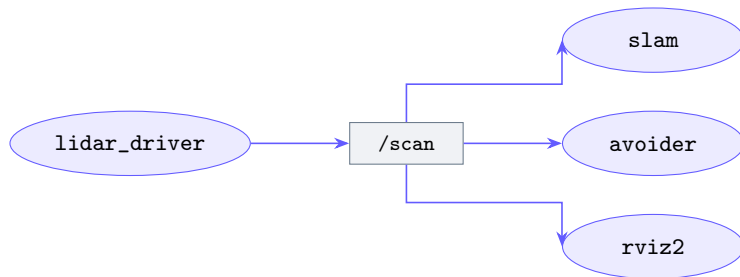
- `robot` — reads sensors, plans, controls, logs, and draws the user interface

It works. Then the planner gets slow, and your control loop goes with it.

The test

If you cannot say what a node does in one short sentence without using “and”, it is two nodes.

Topics: nobody knows who is listening

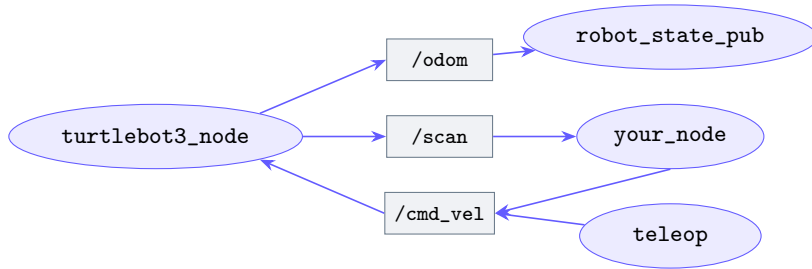


`lidar_driver` publishes and moves on. It does not know three nodes are listening, and behaves identically if none are.

Why this matters more than it looks

Adding a fourth listener needs **no change to the publisher** — which is how you attach a recorder or a visualiser to a running robot without restarting the code that is driving it.

A real graph: TurtleBot3, doing nothing yet



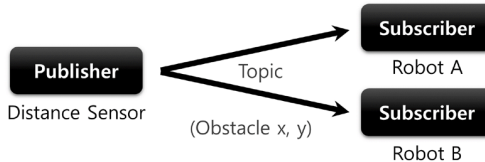
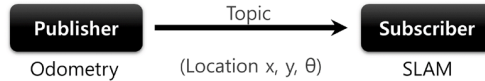
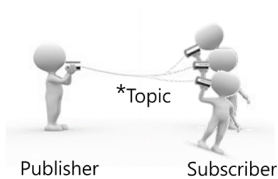
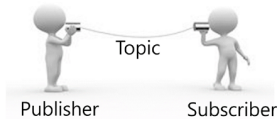
Six processes, started separately, that have never been compiled together. Two of them (`your_node` and `teleop`) publish to the same topic — and the hardware node accepts both, because a topic is many-to-many.

Which is also a bug waiting to happen. Two things commanding one robot is exactly as bad as it sounds. Week 11 is partly about arbitrating that.

Outline

- 1 Nodes and the graph
- 2 Four ways nodes talk
- 3 Messages: the contract
- 4 How nodes find each other
- 5 Quality of Service
- 6 Next

One: a topic — broadcast, and nobody waits

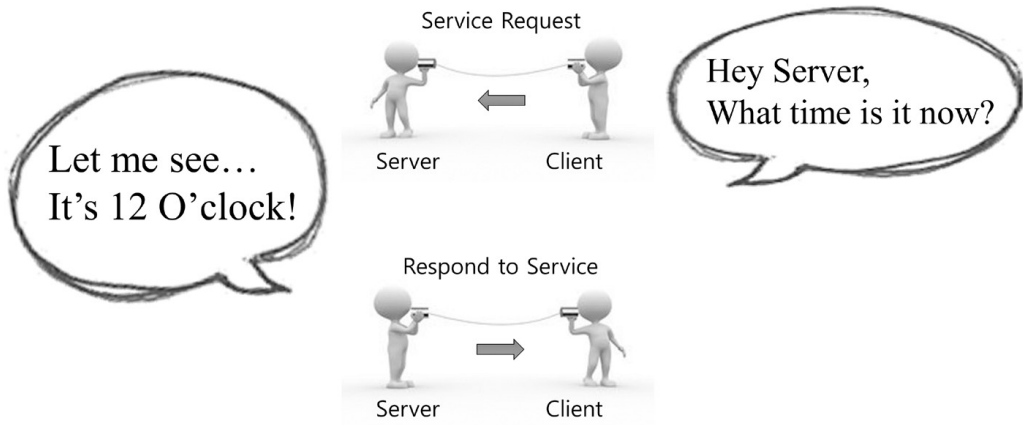


*Topic not only allows 1:1 Publisher and Subscriber communication, but also supports 1:N, N:1 and N:N depending on the purpose.

The publisher does not know who is listening, or whether anyone is. That is the point.

Figure: *ROS Robot Programming*, Y. Pyo, H. Cho, L. Jung, D. Lim (ROBOTIS, 2017). Reused from the instructor's ROS course.

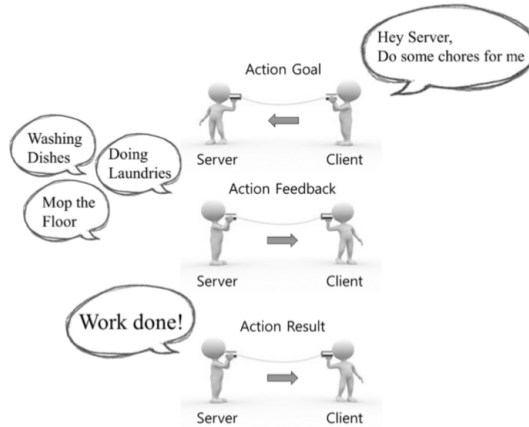
Two: a service — ask, and wait for the answer



One client, one server, one question, one reply. The client blocks until it comes back.

Figure: *ROS Robot Programming*, Y. Pyo, H. Cho, L. Jung, D. Lim (ROBOTIS, 2017). Reused from the instructor's ROS course.

Three: an action — a long job you can watch and cancel



“Mop the floor” takes minutes. You get progress while it runs, and a result at the end.

Figure: *ROS Robot Programming*, Y. Pyo, H. Cho, L. Jung, D. Lim (ROBOTIS, 2017). Reused from the instructor's ROS course.

And four: a parameter

A **parameter** is a named value a node reads at start-up, and that you can change from outside while it runs.

max_speed	0.22	metres per second
scan_topic	/scan	
kp	1.5	the gain in your controller

In the field

The test of whether something is a parameter: **would you want to change it without editing code?** A speed limit, yes. The number of wheels, no. Getting this wrong is why some robot code has forty numbers hard-coded in the middle of a control loop.

The four, side by side

Mechanism	Shape	Use it for	Example
Topic	one-way stream, many-to-many, no-body waits	continuous data	/scan, /odom, /cmd_vel
Service	request → reply, caller waits	a quick question	“reset the odometry”
Action	goal, feedback, result — cancellable	long jobs	“navigate to the door”
Parameter	a named value the node reads, and you can change	configuration	max_speed = 0.22

Topics carry almost all robot traffic — and all of Weeks 4–6. Actions arrive properly in Week 11, when you ask Nav2 to drive somewhere and it takes forty seconds.

How to choose — four questions, in order

❶ **Does it arrive over and over, on its own?**

Sensor readings, velocity commands, poses. $\Rightarrow$ **topic**

❷ **Do I need an answer before I can continue?**

And does it come back immediately? $\Rightarrow$ **service**

❸ **Will it take seconds or minutes, and might I want to stop it?**

$\Rightarrow$ **action**

❹ **Is it a setting, not an event?**

$\Rightarrow$ **parameter**

The mistake this prevents

Using a service for something slow. The caller waits, its callbacks stop running, and the node stops responding to anything else. Every semester somebody builds a `drive_to_goal` service and cannot understand why their robot goes deaf.

Outline

- 1 Nodes and the graph
- 2 Four ways nodes talk
- 3 Messages: the contract
- 4 How nodes find each other
- 5 Quality of Service
- 6 Next

A message is a typed contract

A topic is not a pipe for arbitrary bytes. It carries **one declared type**, and both sides are checked.

`geometry_msgs/msg/Twist`

```
Vector3  linear      # x, y, z   metres / second
Vector3  angular     # x, y, z   radians / second
```

For a differential-drive robot only two of those six numbers mean anything: `linear.x` (forward) and `angular.z` (turn). The rest must be zero — a wheeled robot cannot move sideways.

Under the hood

The light box means **not runnable**. This is an interface definition, compiled into Python and C++ classes at build time. That is why a typo in a field name fails when you build, not at 3 a.m. on the robot.

The types you will meet this semester

Type	Topic	Carries
geometry_msgs/Twist	/cmd_vel	Commanded v and ω
sensor_msgs/LaserScan	/scan	One LiDAR sweep: ranges, angles, timestamps
nav_msgs/Odometry	/odom	Estimated pose and velocity, with covariance
sensor_msgs/Imu	/imu	Angular rate, linear acceleration
nav_msgs/OccupancyGrid	/map	The map, as a grid of cells

In the field

These are not this course's conventions — they are the field's. A robot from a different vendor publishes `sensor_msgs/LaserScan` on `/scan`. Learn the type once, and your obstacle-avoidance node works on hardware you have never seen.

Every message is stamped, and that is not a detail

Most sensor messages carry a `std_msgs/Header`: a timestamp and the name of the coordinate frame the data is expressed in.

<code>stamp</code>	when the measurement was <i>taken</i> — not when it arrived
<code>frame_id</code>	which frame it is measured in — <code>base_link</code> , <code>laser</code> , <code>map</code>

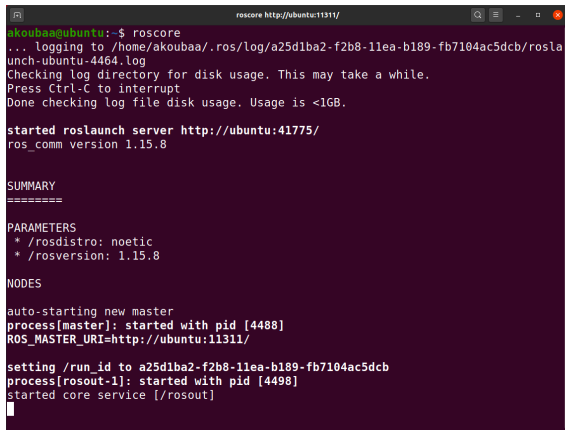
Truth 2, in a struct

Last week: nothing is instantaneous. The stamp is how a robot copes — it lets a node fuse a scan taken 100 ms ago with odometry taken 20 ms ago, *correctly*. Week 7 (TF2) and Week 9 (estimation) are built entirely on these two fields.

Outline

- 1 Nodes and the graph
- 2 Four ways nodes talk
- 3 Messages: the contract
- 4 How nodes find each other**
- 5 Quality of Service
- 6 Next

This process does not exist any more

A terminal window titled 'roscore http://ubuntu:11311/' showing the output of the 'roscore' command. The output includes logging information, a disk usage check, and the successful start of the roslaunch server. It also displays a summary of parameters and nodes.

```
akoubaa@ubuntu:~$ roscore
... logging to /home/akoubaa/.ros/log/a25d1ba2-f2b8-11ea-b189-fb7104ac5dcb/ros-launch-ubuntu-4464.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://ubuntu:41775/
ros_comm version 1.15.8

SUMMARY
=====

PARAMETERS
* /rostdistro: noetic
* /rosversion: 1.15.8

NODES

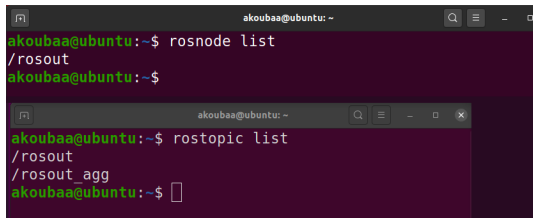
auto-starting new master
process[master]: started with pid [4488]
ROS_MASTER_URI=http://ubuntu:11311/

setting /run_id to a25d1ba2-f2b8-11ea-b189-fb7104ac5dcb
process[rosout-1]: started with pid [4498]
started core service [/rosout]
```

In ROS 1 nothing connected until roscore was running. ROS 2 Jazzy deleted it.

Screenshot: the instructor's ROS 1 course (2020), ROS Noetic. Shown as history, not as something you will run.

Half of what you will find online looks like this



```
akoubaa@ubuntu: ~  
akoubaa@ubuntu:~$ roscore  
/rosout  
akoubaa@ubuntu:~$  
akoubaa@ubuntu:~$ rostopic list  
/rosout  
/rosout_agg  
akoubaa@ubuntu:~$
```

Screenshot: the instructor's ROS 1 course (2020), ROS Noetic.

One tool, `ros2`, with the noun second. **This course is ROS 2 Jazzy only** — but ROS 1 ran for fifteen years, and most of the tutorials, forum answers and GitHub code you will find were written for it.

How to tell in one second: if it starts with `ros` and has no space after it, it is ROS 1.

ROS 1	ROS 2 Jazzy
<code>roscore</code>	— gone
<code>roscore list</code>	<code>ros2 node list</code>
<code>rostopic list</code>	<code>ros2 topic list</code>
<code>roscore</code>	<code>ros2 run</code>
<code>roslaunch</code>	<code>ros2 launch</code>
<code>roscore show</code>	<code>ros2 interface show</code>
<code>catkin_make</code>	<code>colcon build</code>

There is no central process

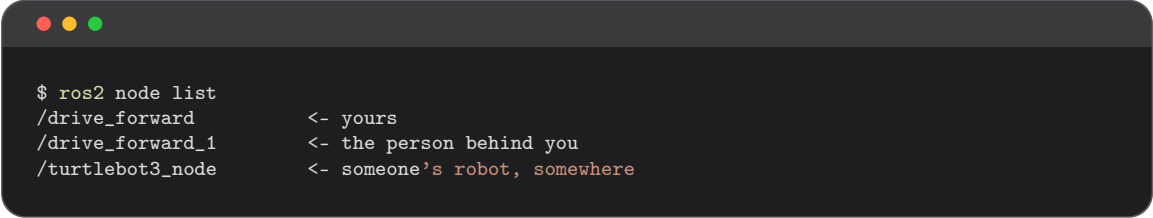
When a node starts, it announces itself on the network and listens for others doing the same. Publishers and subscribers on the same topic then connect **directly**.

	ROS 1	ROS 2
Finding each other	A central roscore kept the directory	Distributed discovery — no directory
If that dies	Nothing new can connect. The robot goes deaf.	There is nothing to die
Start-up order	Master first, always	Any order
Delivery guarantees	One behaviour, take it or leave it	Chosen per topic (next section)

This is the main reason ROS 2 exists, and the main reason it appears in products where ROS 1 largely did not. It is a rewrite, not an upgrade.

The consequence you met in Week 1

Discovery is **automatic** and reaches everyone on the network segment. In a shared lab, that is a problem:



```
$ ros2 node list
/drive_forward          <- yours
/drive_forward_1        <- the person behind you
/turtlebot3_node         <- someone's robot, somewhere
```

ROS_DOMAIN_ID partitions discovery. Same ID $\Rightarrow$ you see each other.

First diagnostic question of the entire semester: a topic is obviously being published and your node sees nothing — are you on the same domain ID?

Outline

- 1 Nodes and the graph
- 2 Four ways nodes talk
- 3 Messages: the contract
- 4 How nodes find each other
- 5 Quality of Service**
- 6 Next

The setting everyone skips, and then loses a day to

A publisher and a subscriber must agree on **how** messages are delivered, not just on the type. That agreement is the QoS profile.

Setting	Options	What it decides
Reliability	RELIABLE / BEST_EFFORT	Retry lost messages, or let them go
Durability	VOLATILE / TRANSIENT_LOCAL	Does a late subscriber get the last message, or nothing until the next one?
History + depth	KEEP_LAST(n) / KEEP_ALL	How many to hold while the subscriber is busy

These are not performance knobs. They change whether your data arrives at all.

Choosing: it depends what the data is for

LiDAR scans — BEST_EFFORT

10 per second. If one is lost, another arrives in 100 ms.

Retrying a stale scan is worse than useless: it delays the fresh one behind it.

A map — TRANSIENT_LOCAL

Published once, rarely.

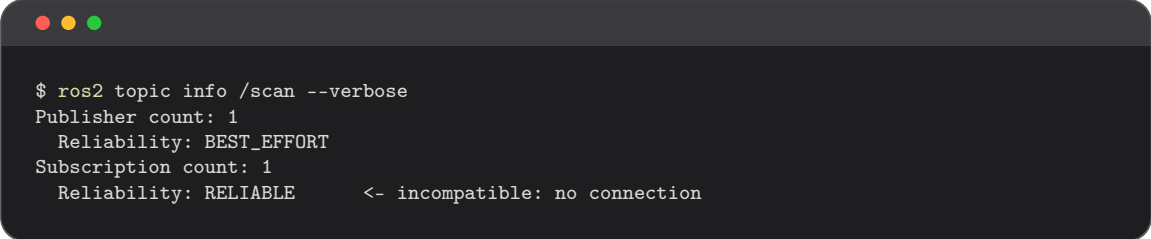
A node that starts late must still receive it — otherwise it waits forever for a message that already happened.

The rule of thumb

Streams of sensor data: best-effort, shallow queue. **State** that is published once and must not be missed: reliable and transient-local. Commands (`/cmd_vel`): reliable — a dropped stop command is a crash.

The failure that looks like nothing at all

Publisher is BEST_EFFORT. Subscriber demands RELIABLE. Both start cleanly, report no errors, and **nothing is ever delivered**.



```
$ ros2 topic info /scan --verbose
Publisher count: 1
  Reliability: BEST_EFFORT
Subscription count: 1
  Reliability: RELIABLE      <- incompatible: no connection
```

The subscriber asked for a guarantee the publisher does not offer, so the two are never matched. Not a bug — the system refusing to pretend.

The command that answers it

```
ros2 topic info /scan --verbose
```

The ROS 2 command-line tool. Every command in this course starts with it.

The command that answers it

```
ros2 topic info /scan --verbose
```

The thing you are asking about: topics.
Others are node, service, param, bag.

The command that answers it

```
ros2 topic info /scan --verbose
```

|
What to do with it. `list`, `echo`,
`hz`, `pub` are the others you will use.

The command that answers it

```
ros2 topic info /scan --verbose
```

Which topic. The leading slash is part of the name.

The command that answers it

```
ros2 topic info /scan --verbose
```

Without this flag you do not see the QoS profiles — and the QoS profiles are the whole answer.

Silent topic: the order to check, every time

- 1 `ros2 node list` — **is the publisher even alive?**
- 2 `ros2 topic list` — does the topic exist, spelled exactly that way?
- 3 `ros2 topic info <topic> --verbose` — counts first, then QoS.
- 4 `printenv ROS_DOMAIN_ID` — **in both terminals.**

Publisher 1, subscriber 0

Both 1, no data

Topic missing entirely

Different topic name, or different domain

QoS mismatch — read the two profiles

Publisher not running, or shell not sourced

Learn this order now. It is the same four commands in Week 11, on a robot with forty topics.

Outline

- 1 Nodes and the graph
- 2 Four ways nodes talk
- 3 Messages: the contract
- 4 How nodes find each other
- 5 Quality of Service
- 6 Next

What session B does with all of this

- 1 Run a robot simulator *somebody else wrote*, and take its graph apart from the command line.
- 2 Build a workspace and a package of your own.
- 3 Write a publisher, then a subscriber, and watch them find each other.
- 4 Start both with a single launch file.
- 5 Break it on purpose, five ways, and fix each.

Bring a laptop with the setup check passing. Without it, you will watch.

Quiz 1 today — nodes, topics, the four mechanisms, and one QoS question.